

Beginner's Guide to Building a Complete Application Using AI

A practical, non-technical playbook for turning an idea into a working app using ChatGPT, Claude, Gemini, Codex, Claude Code, Google AI Studio, Firebase Studio, or Google Antigravity

Created for beginners, founders, creators, and small business owners

Version: May 2026

Source code used in the video: DharmaPath Application

Highlighted URL: <https://github.com/Amanshakya-code/DharmaPath>

How to use this document: This guide is designed for people who do not know coding but want to use AI to create websites, business apps, SaaS tools, internal dashboards, booking systems, marketplaces, and other digital products. The goal is not to become a developer overnight. The goal is to learn how to give AI the right instructions, validate the output, test carefully, and launch safely.

Table of Contents

1. Mindset: what AI can and cannot do for non-technical builders
2. Choose your AI app-building route
3. The complete workflow from idea to launch
4. Step-by-step guide for beginners
5. Prompt library: copy, paste, and use
6. Requirement checklist: what the AI must ask before building
7. Example: turning a business idea into an app plan
8. Testing, security, deployment, and launch checklist
9. Common mistakes and how to avoid them
10. Appendix: one master prompt for the complete process
11. Reference links

1. Mindset: What AI Can and Cannot Do for Non-Technical Builders

AI tools can now help a beginner move from an app idea to a working prototype much faster than before. You can describe the product in plain English, ask the AI to interview you, generate a plan, create screens, write code, test flows, fix bugs, and prepare deployment steps. The key is to work in small, controlled phases instead of asking AI to build everything at once.

Reality check: AI can generate the app, but you are still responsible for the business idea, user experience decisions, testing, legal compliance, privacy, payments, and launch quality. For real customer data, payments, healthcare, legal, finance, or sensitive workflows, get a technical/security review before launch.

What a non-technical person can realistically build

- **Landing pages and portfolio websites:** business pages, service websites, creator pages, lead capture pages, and product pages.
- **Internal tools:** admin dashboards, approval workflows, inventory trackers, CRM-like tools, reporting dashboards, and employee portals.
- **Simple SaaS products:** subscription-based tools, AI content tools, resume builders, invoice tools, job trackers, habit trackers, and niche productivity apps.
- **Marketplaces and booking apps:** appointment booking, class booking, event listing, vendor listing, and directory apps.
- **AI-powered apps:** chatbots, document analyzers, content generators, business assistants, and support tools using Gemini, OpenAI, Claude, or similar APIs.

What you should not launch without expert help

- Apps that handle medical advice, diagnosis, financial trading, tax filing, legal advice, or high-risk personal decisions.
- Apps that store sensitive identity documents, passwords, payment card details, or confidential business information.
- Apps where downtime, wrong calculations, or data loss could harm customers or businesses.
- Apps that need complex compliance, such as HIPAA, GDPR-heavy workflows, PCI-DSS payment handling, banking, or insurance.

2. Choose Your AI App-Building Route

Before writing prompts, choose the route you want to follow. For a non-technical person, the easiest route is usually: use ChatGPT, Claude, or Gemini to clarify the requirements and create a plan, then paste that plan into an AI coding tool such as Codex, Claude Code, Firebase Studio, Google Antigravity, Cursor, Replit, or another builder.

Recommended beginner routes

Route	Best for	How it works	Beginner rating
ChatGPT + Codex	People who want planning plus agentic coding	Use ChatGPT for requirements and PRD. Use Codex/Codex CLI/IDE extension to create or edit the project files.	Good
Claude + Claude Code	People who want strong codebase understanding and	Use Claude for product planning. Use Claude Code to	Good

	iterative implementation	create features, fix bugs, run commands, and work across files.	
Gemini + Google AI Studio	People building AI-powered apps with Gemini APIs	Use Gemini for planning and Google AI Studio for Gemini API exploration, API keys, and model testing.	Medium
Firebase Studio App Prototyping Agent	Fast web app prototypes	Describe the app in natural language. The agent can generate a blueprint, code, and preview. Note: plan migration because Firebase Studio is scheduled to sunset in 2027.	Very good for prototypes
Google Antigravity	Agent-first development with Google ecosystem	Use agents across editor, terminal, and browser to plan, execute, and verify coding tasks.	Medium
No-code builder + AI planner	Landing pages, simple workflows, and quick validation	Use AI for product requirements and copy. Use no-code tools for build and launch.	Very good

Simple default choice: If you are confused, start with this simple stack: ChatGPT or Claude for planning + Firebase Studio or Codex/Claude Code for building + Firebase/Supabase for database and authentication + Vercel/Netlify/Firebase Hosting for deployment.

Important naming clarification

You mentioned Gemini, ChatGPT, “cloud,” “Codis,” “clones CLI,” and Google Flow. In this document, these are normalized as follows:

- “Cloud” is treated as Claude by Anthropic, especially Claude Code for coding workflows.
- “Codis” or “CLI” is treated as Codex / Codex CLI when referring to OpenAI’s coding agent.
- “Google Flow” is treated as Google’s AI app-building ecosystem, including Google AI Studio, Firebase Studio, and Google Antigravity. Google naming changes over time, so always check the current official product page before starting a new project.

3. The Complete Workflow from Idea to Launch

The safest workflow is not “ask AI to build the full app.” The safest workflow is “ask AI to ask questions first, then generate a plan, then build the app in small phases.”

1. Write your raw app idea in simple English.
2. Use Prompt 1 to make the LLM interview you and collect requirements.
3. Answer the AI’s questions about users, screens, data, UI, business model, and app behavior.
4. Use Prompt 2 to convert those answers into a Product Requirements Document (PRD), feature list, screen list, user journey, and MVP plan.
5. Use Prompt 3 to convert the PRD into an implementation plan for the chosen coding tool.

6. Paste the implementation plan into Codex, Claude Code, Firebase Studio, Google Antigravity, or your preferred coding agent.
7. Build only the first version of the MVP first: authentication, core screens, database, and one or two core workflows.
8. Test manually like a real user. Take screenshots of bugs and paste them into the AI with clear instructions.
9. Add payments, admin panel, analytics, email notifications, and AI features only after the core app works.
10. Deploy to a staging environment, test again, then launch to real users.

One-page workflow diagram in words

```
graph TD; A[Idea] --> B[Requirement Interview Prompt]; B --> C[User answers all questions]; C --> D[PRD + MVP Plan]; D --> E[Technical Implementation Plan]; E --> F[AI Coding Agent builds Phase 1]; F --> G[Manual testing + bug screenshots]; G --> H[AI fixes issues]; H --> I[Deploy staging]; I --> J[Security/payment review]; J --> K[Launch MVP]; K --> L[Collect feedback and improve];
```

Idea
↓
Requirement Interview Prompt
↓
User answers all questions
↓
PRD + MVP Plan
↓
Technical Implementation Plan
↓
AI Coding Agent builds Phase 1
↓
Manual testing + bug screenshots
↓
AI fixes issues
↓
Deploy staging
↓
Security/payment review
↓
Launch MVP
↓
Collect feedback and improve

4. Step-by-Step Guide for Beginners

Step 1: Pick one app idea and one target user

Do not start with “I want to build an app that does everything.” Start with a very specific target user and one painful problem. The more specific the problem, the better AI can design the product.

Example

Bad idea:
Create a business app for everyone.

Better idea:

Create a booking and payment website for home salon service providers in Delhi NCR. The app should allow customers to choose a service, select a time slot, pay online, and receive WhatsApp/email confirmation.

Step 2: Choose the build type

- Website only: best for portfolio, landing page, blog, service pages, business website, and lead generation.
- Web application: best for dashboards, login-based apps, SaaS products, booking apps, and admin panels.
- Mobile app: best when users need phone-first experience, notifications, camera, GPS, offline access, or app-store distribution.
- AI app: best when the main value is generated by an AI model, such as summarization, chat, generation, classification, extraction, or automation.

Step 3: Use AI as a product manager first, not a coder first

The first LLM prompt should not ask for code. It should ask the AI to interview you. This prevents missing screens, broken assumptions, weak UI, unclear user roles, and incomplete flows.

Prompt 1: Requirement Interviewer

You are a senior product manager, UX researcher, and software architect.
I am a non-technical founder. I want to create an application using AI coding tools.
Do not generate code yet.
Your job is to interview me and collect all requirements needed to build the application.

Ask me questions in sections:

1. App idea and business goal
2. Target users and user roles
3. Core problem and value proposition
4. Main features for MVP
5. Screens/pages required
6. User journeys and workflows
7. Login, signup, and permissions
8. Data that needs to be stored
9. Admin panel requirements
10. UI style, colors, branding, and design inspiration
11. Payment or subscription requirements
12. Notifications: email, SMS, WhatsApp, push
13. AI features, if any
14. Reports, analytics, and dashboard requirements
15. Security, privacy, and compliance concerns
16. Deployment preference and budget
17. Nice-to-have features for future versions

Ask only the questions first. Wait for my answers.

After I answer, summarize my requirements and ask follow-up questions if anything is unclear.

Step 4: Answer the questions in a structured way

When AI asks questions, do not answer casually with one line. Use a structured answer. You can copy this template and fill it.

Requirement Answer Template

App name:
One-line idea:
Target users:
User roles:
Main problem it solves:
Top 5 MVP features:
Pages/screens needed:
User workflow from start to finish:
Admin workflow:
Data to store:
Login/signup needed:
Payment needed:
Notifications needed:
AI features needed:
UI style:
Color preference:
Competitor/reference websites:
Budget preference:
Launch deadline:
Future features:
Anything I definitely do not want:

Step 5: Ask the LLM to generate a plan

After the requirements are collected, ask the LLM to create a clear product plan. This plan becomes the main document that you paste into the coding agent.

Prompt 2: Generate the Product Plan

Based on my answers, create a complete Product Requirements Document for this application.
Do not write code yet.

The document must include:

1. App summary
2. Target users
3. User roles and permissions
4. MVP feature list
5. Future feature list
6. Full screen/page list
7. Detailed user journeys
8. Admin journeys
9. Database entities in simple language
10. APIs or backend actions in simple language

11. UI/UX direction
12. Recommended tech stack for a beginner
13. Development phases
14. Testing checklist
15. Deployment checklist
16. Risks and assumptions
17. Questions still open

Keep the language beginner-friendly but detailed enough that I can paste it into an AI coding agent.

Step 6: Convert the product plan into a coding-agent task

A coding agent needs implementation instructions. It should know the stack, phases, file structure, and quality expectations. Use the next prompt to convert the plan into a developer-style execution plan.

Prompt 3: Convert Plan to Coding-Agent Brief

Convert the Product Requirements Document into a coding-agent implementation brief.

The output will be pasted into an AI coding tool such as Codex, Claude Code, Firebase Studio, Google Antigravity, Cursor, or Replit.

Include:

1. Recommended tech stack
2. Project setup steps
3. Folder structure
4. Environment variables needed
5. Database schema
6. Authentication approach
7. Screen-by-screen implementation tasks
8. API/backend tasks
9. UI component list
10. Validation rules
11. Error states and empty states
12. Test cases
13. Build order in phases
14. Definition of done for each phase
15. Exact first task the coding agent should start with

Important: Tell the coding agent to build incrementally and stop after each phase for review.

Step 7: Paste into your selected coding tool

Now open the coding tool and paste the implementation brief. Do not paste all future features at once. Start with Phase 1: project setup, core layout, authentication, database, and the main workflow.

Tool	What to paste first	How to work safely
Codex / Codex CLI	The coding-agent brief and Phase 1 task	Ask it to create a new project, explain changed files, run tests/builds, and stop

		for review.
Claude Code	The coding-agent brief and specific first implementation task	Ask it to inspect files, create a plan, make small commits/changes, and verify the build.
Firebase Studio	Natural-language app description plus PRD summary	Use the generated blueprint and preview. Review screens before adding payments or sensitive data.
Google Antigravity	Task-oriented brief with verification expectations	Ask the agent to plan, execute, verify, and report what changed.
Google AI Studio	Gemini API prompts or AI-feature prototype prompts	Use it to test AI behavior, generate API examples, and design model prompts before app integration.

Step 8: Build in phases

- **Phase 1: Foundation:** project setup, design system, layout, navigation, database, authentication.
- **Phase 2: Core workflow:** the one main thing users must do in the app, such as booking, generating a report, submitting a request, or creating content.
- **Phase 3: Admin panel:** admin dashboard, user management, records, status changes, reports, and basic controls.
- **Phase 4: Notifications and payments:** email, WhatsApp/SMS, payment checkout, invoices, subscriptions, and receipts.
- **Phase 5: Polish and launch:** responsive UI, empty states, error handling, loading states, analytics, security rules, backups, and deployment.

Step 9: Test manually before adding more features

Every time the AI builds a phase, behave like a user and test the app. Click every button. Fill every form. Try wrong inputs. Try mobile view. If something breaks, take a screenshot and paste the issue into the AI.

Prompt: Bug Fix with Screenshot/Details

The app has this issue: [describe the issue].

Expected behavior: [what should happen].

Actual behavior: [what happened].

Steps to reproduce:

1. [step]
2. [step]
3. [step]

Please inspect the code, identify the root cause, fix it, and explain which files you changed.

After fixing, run the build/test command and tell me whether it passed.

Step 10: Launch only after a checklist review

Do not launch just because the app “looks good.” Launch after you have tested the happy path, failure paths, mobile view, security rules, authentication, payment flow, email flow, and data backup plan.

5. Prompt Library: Copy, Paste, and Use

This section contains ready-to-use prompts. Replace the text inside square brackets with your own details. Use them in sequence. Do not skip the requirement prompts.

Prompt 4: Choose the Best Build Path

Act as my AI app-building mentor.

I am non-technical and want to build an app using AI tools.

First, help me decide whether I should build:

1. A landing page
2. A web application
3. A mobile app
4. An AI-powered app
5. A no-code MVP

Ask me 10 questions about my idea, target users, budget, deadline, and technical comfort.

Then recommend the simplest build path and explain why.

Prompt 5: UI/UX Design Brief

Act as a senior UX/UI designer.

Create the UI/UX requirements for my application based on the product plan below.

Product plan:

[paste plan here]

Include:

1. Design style and personality
2. Color palette recommendation
3. Typography recommendation
4. Layout principles
5. Navigation structure
6. Screen-by-screen wireframe description
7. Components needed
8. Mobile responsive behavior
9. Loading, empty, error, and success states
10. Accessibility considerations
11. Microcopy for buttons, labels, and messages
12. What the first version should not include

Make the UI modern, clean, fast, and easy for non-technical users.

Prompt 6: Backend and Database Plan

Act as a backend architect.

Based on the product requirements below, design the backend in beginner-friendly language.

Requirements:

[paste PRD here]

Provide:

1. User roles and permissions
2. Database tables/entities
3. Fields for each table
4. Relationships between tables
5. Backend actions/API list
6. Validation rules
7. Authentication and password reset flow
8. Admin-only actions
9. Audit/logging requirements
10. Security rules and data access rules
11. Backup strategy
12. Simple explanation of how data moves through the app

Do not write code yet. Make this ready for a coding agent.

Prompt 7: First Coding-Agent Prompt

You are an AI coding agent.

Build this application incrementally.

Rules:

1. Read the full brief first.
2. Create a short implementation plan before coding.
3. Build only Phase 1 first.
4. Do not add future features yet.
5. Use clean, maintainable code.
6. Add clear file names and folder structure.
7. Add form validation and basic error handling.
8. Make the UI responsive for mobile and desktop.
9. After implementation, run the build/test command.
10. Explain what was changed and what I should review.

Application brief:

[paste coding-agent brief here]

Start with Phase 1 only.

Prompt 8: QA Test Plan

Review the current application like a senior QA tester.

Create a detailed manual test plan.

Include tests for:

1. Signup/login/logout
2. Each user role
3. Each screen
4. Each form
5. Required fields
6. Wrong inputs
7. Empty states
8. Error states
9. Mobile responsiveness
10. Admin workflows
11. Payment/subscription flow, if any
12. Email/SMS/WhatsApp notifications, if any
13. Security checks
14. Data privacy checks
15. Final launch readiness

Give the output as a checklist I can follow manually.

Prompt 9: Deployment Guide

Act as a deployment engineer.

I am non-technical. Explain how to deploy this app safely.

Project details:

[paste tech stack and app summary]

Include:

1. Recommended hosting platform
2. Required accounts
3. Environment variables
4. Database setup
5. Domain setup
6. SSL/HTTPS setup
7. Build command
8. Deployment command or steps
9. How to test staging
10. How to rollback
11. How to monitor errors
12. Common deployment mistakes

Explain each step in simple language.

Prompt 10: Revenue and Pricing Plan

Act as a SaaS business strategist.

Based on my app idea and target users, suggest how this app can generate revenue.

App idea:

[paste summary]

Include:

1. Possible pricing models
2. Free vs paid feature split
3. Subscription plans
4. One-time payment option
5. Trial strategy
6. Upsell/cross-sell ideas
7. What to avoid in pricing
8. First 10 customers strategy
9. Landing page copy angle
10. Metrics to track after launch

Keep suggestions practical for a solo founder or small business.

Prompt 11: Security Review

Act as a security reviewer for a beginner-built AI application.

Review the app plan below and identify security and privacy risks.

App plan:

[paste PRD or technical brief]

Check for:

1. Sensitive data collection
2. Authentication risks
3. Permission issues
4. Database access rules
5. File upload risks
6. Payment risks
7. API key exposure
8. Prompt injection risks for AI features
9. Logging sensitive data
10. Backup and deletion policy
11. Terms/privacy policy needs
12. What must be reviewed by a professional before launch

Give me a prioritized action list.

6. Requirement Checklist: What the AI Must Ask Before Building

A strong requirements phase prevents 80% of beginner mistakes. Use this checklist to make sure the AI collected enough information before generating a plan or code.

Business and user requirements

- Who is the target user?
- What problem does the app solve?
- Why will users pay or use it repeatedly?
- What is the first version/MVP?
- What features should be delayed?
- What is the success metric: signups, bookings, revenue, leads, time saved, reports generated?

Screen requirements

- Landing page / home page
- Signup and login pages
- Forgot password page
- User dashboard
- Main workflow screen
- Profile/settings page
- Payment/subscription page, if needed
- Admin dashboard
- Record details page
- Reports/analytics page
- Help/support/contact page
- Terms and privacy pages

UI requirements

- Should the UI look premium, playful, corporate, minimal, colorful, luxury, startup-style, or enterprise-style?
- Should the app be mobile-first or desktop-first?
- What colors should be used or avoided?
- Are there reference websites/apps to match?
- Should the UI include animations?
- Should the UI support dark mode?
- What should the empty state say when there is no data?
- What should the loading state look like?
- What should happen when an error occurs?

Technical and data requirements

- What information must be stored for each user?
- Which users can see which records?
- Is admin approval required?
- Are file uploads needed?
- Are payments needed?
- Are notifications needed?
- Is search/filter/sorting needed?
- Is export to CSV/PDF needed?
- Is AI integration needed?
- What reports or dashboards are needed?

Launch requirements

- Where will the app be hosted?
- What domain will be used?
- Who will maintain it?
- How will bugs be reported?
- How will backups be handled?
- How will users contact support?
- What are the terms, privacy, and refund policies?
- What analytics will track user behavior?

7. Example: Turning a Business Idea into an App Plan

Raw idea

I want to build an app for local fitness trainers.

They should be able to create workout plans for clients, collect payments, track progress, and send reminders.

Better requirement answer

App name: FitCoach Manager

One-line idea: A web app where independent fitness trainers can manage clients, workout plans, progress, payments, and reminders.

Target users: Independent personal trainers and small gyms.

User roles: Trainer, Client, Admin.

Main problem: Trainers manage clients using WhatsApp and spreadsheets, which becomes messy.

Top MVP features:

1. Trainer signup/login
2. Client management
3. Workout plan creation
4. Progress tracking
5. Payment status tracking
6. Client dashboard
7. Reminder emails

Pages/screens:

Home, pricing, login, trainer dashboard, client list, client profile, workout plan builder, progress tracker, payment status, client dashboard, admin panel.

UI style: Modern, clean, fitness-focused, mobile-friendly, energetic but not too colorful.

Payment: Start with manual payment status; add online payments later.

Launch deadline: MVP in 2 weeks.

Example phased build plan

Phase	Goal	Features	Stop condition
1	Foundation	Project setup, responsive layout, auth, trainer dashboard shell	Trainer can sign up and see dashboard
2	Client management	Add/edit/delete clients, client	Trainer can manage clients

		profile, client status	
3	Workout plans	Create workout plan, assign to client, view plan	Client can see assigned plan
4	Progress tracking	Add progress entries, charts/table view	Trainer can track progress
5	Launch polish	Emails, mobile polish, empty states, deployment	MVP ready for pilot users

8. Testing, Security, Deployment, and Launch Checklist

Manual testing checklist

- Create a new user account.
- Login with correct password.
- Try login with wrong password.
- Reset password if the feature exists.
- Create, edit, and delete one main record.
- Try submitting empty forms.
- Try very long text in form fields.
- Open the app on mobile, tablet, and desktop sizes.
- Check all buttons and links.
- Refresh the page during the main workflow.
- Logout and confirm protected pages cannot be opened.
- Test admin-only pages with a normal user account.
- Check email/payment/notification flows, if included.

Security basics for beginners

- Never paste API keys, database passwords, private tokens, or production credentials into public chats.
- Use environment variables for secrets.
- Do not store payment card numbers yourself. Use a trusted payment provider.
- Make sure users can only access their own data.
- Use HTTPS for production.
- Use strong authentication and password reset from a trusted provider.
- Add database backups before launch.
- Add a privacy policy if collecting user data.
- Use role-based permissions for admin features.
- Ask a developer/security reviewer to inspect the app before handling real customer data.

Deployment checklist

- Staging deployment works.
- Production environment variables are configured.
- Database is connected to production environment.
- Domain and SSL are working.
- Authentication callback URLs are configured.
- Payment webhook URLs are configured, if needed.
- Error monitoring is enabled.
- Analytics are enabled.

- Backup and rollback plan exists.
- A test user can complete the core workflow end-to-end.

Launch checklist

- Landing page clearly explains the product.
- Pricing is simple.
- Support email or contact method is visible.
- Terms/privacy pages are available.
- First 10 pilot users are identified.
- Feedback form is ready.
- You know which metric you will track after launch.
- You have a plan for fixing bugs quickly.

9. Common Mistakes and How to Avoid Them

Mistake	Why it hurts	Better approach
Asking AI to build the entire app in one prompt	The app becomes messy, incomplete, or hard to debug.	Ask for requirements first, then PRD, then phased implementation.
Skipping UI requirements	The app may work but look unprofessional.	Give UI style, references, colors, target user, and screen expectations.
Adding payments too early	Payment bugs can hurt trust and create legal/financial issues.	Build the core workflow first, then add payments after testing.
Not testing mobile view	Many users may open the app on phones.	Test every major screen on mobile width.
No admin panel	You cannot manage users, records, refunds, or support requests.	Include admin dashboard requirements early.
No backup plan	Data loss can destroy user trust.	Ask AI to create backup and rollback steps.
Using real customer data during testing	Privacy and security risk.	Use fake data until production is reviewed.
Copying code without understanding basics	You cannot debug or explain issues.	Ask AI to explain changed files and commands after every phase.

How to talk to the AI when you are stuck

Prompt: Explain and Fix for Beginners

I am non-technical. Explain this issue in simple language.

Tell me:

1. What went wrong
2. Why it happened
3. Which file is responsible
4. What you changed
5. How I can test the fix
6. How to avoid this issue later

Here is the error/message/screenshot:

[paste error or upload screenshot]

10. Appendix: One Master Prompt for the Complete Process

Use this when you want one big prompt that starts with requirements collection and ends with a coding-agent-ready plan. The AI should not code immediately. It should interview you first.

Master Prompt: Complete AI App Builder

You are my complete AI app-building mentor, product manager, UX designer, software architect, QA tester, and launch advisor.

I am a non-technical person. I want to create a real application using AI tools such as ChatGPT, Claude, Gemini, Codex, Claude Code, Google AI Studio, Firebase Studio, Google Antigravity, Cursor, Replit, or similar tools.

Important rules:

1. Do not generate code in the beginning.
2. First interview me and collect all requirements.
3. Ask clear questions in sections.
4. Explain everything in beginner-friendly language.
5. After I answer, summarize my requirements.
6. Ask follow-up questions if anything is unclear.
7. Then create a Product Requirements Document.
8. Then create a UI/UX plan.
9. Then create a backend/database plan.
10. Then create a phased implementation plan for an AI coding agent.
11. Then create testing, deployment, security, and launch checklists.
12. Build the app in phases, not all at once.

Start by asking me questions for:

- App idea and business goal
- Target users
- User roles
- Main problem
- MVP features
- Future features
- Screens/pages
- User journeys
- Admin journeys
- Data to store
- Login and permissions
- Payments/subscriptions
- Notifications
- AI features
- UI style and brand preferences
- Competitor/reference apps
- Reports/analytics
- Security/privacy requirements
- Deployment and budget
- Launch deadline

After I answer, produce:

1. Requirement summary
2. PRD
3. Screen list
4. User journey map
5. Admin workflow
6. UI/UX brief
7. Database plan
8. API/backend plan
9. Suggested tech stack
10. Phased implementation plan
11. First coding-agent prompt
12. Manual testing checklist
13. Deployment checklist
14. Launch checklist
15. Revenue/pricing suggestions

Begin by asking the requirement questions only.

11. Tool-Specific Starter Prompts

For Codex / Codex CLI

You are working inside my project directory.
Read the project files first and summarize the current structure.
Then implement only the task below.
Before coding, create a short plan.
After coding, run the build/test command and summarize changed files.

Task:

[paste Phase 1 task or bug fix here]

Constraints:

- Keep changes small and reviewable.
- Do not add unrelated features.
- Do not expose secrets.
- Stop after completing this task.

For Claude Code

Please inspect the codebase and understand the current app.
Then create a plan to implement this feature safely.
Ask before making any destructive changes.
Implement the smallest working version first.
Run relevant tests/builds.

Explain changed files and how I can verify the feature.

Feature:

[paste feature here]

For Gemini / Google AI Studio

I am designing an AI feature for my application.

Help me create the best prompt and response format for Gemini.

Feature goal:

[paste goal]

User input:

[paste example]

Desired output:

[paste desired output]

Please provide:

1. System instruction
2. User prompt template
3. JSON response schema
4. Safety rules
5. Example input and output
6. Edge cases to test

For Firebase Studio / App Prototyping Agent

Create a full-stack web app prototype based on this product brief.

Build the MVP only.

Focus on clean UI, responsive design, authentication, database-backed core workflow, and a preview I can test.

Product brief:

[paste PRD summary]

MVP screens:

[paste screen list]

Core workflow:

[paste workflow]

Design style:

[paste UI style]

Stop after the MVP prototype and tell me what to review before adding payments or advanced features.

For Google Antigravity

Use agent mode to complete this task end-to-end.

First plan, then execute, then verify.

Use the editor, terminal, and browser preview where needed.

Task:

[paste task]

Verification requirements:

1. App builds successfully
2. Main workflow works in browser preview
3. UI is responsive
4. No obvious console errors
5. Explain changed files and remaining risks

Do not work on future phases yet.

12. Reference Links

Official references used to keep tool names and workflows aligned with current product documentation:

- OpenAI Codex: <https://developers.openai.com/codex>
- OpenAI Codex CLI: <https://developers.openai.com/codex/cli>
- OpenAI Codex IDE extension: <https://developers.openai.com/codex/ide>
- Claude Code overview: <https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview>
- Claude Code quickstart: <https://docs.anthropic.com/en/docs/claude-code/quickstart>
- Google AI Studio: <https://ai.google.dev/aistudio>
- Gemini API documentation: <https://ai.google.dev/gemini-api/docs>
- Firebase Studio documentation: <https://firebase.google.com/docs/studio>
- Firebase Studio App Prototyping Agent: <https://firebase.google.com/docs/studio/get-started-ai>
- Firebase Studio sunset and migration: <https://firebase.google.com/docs/studio/migrating-project>
- Google Antigravity: <https://antigravity.google/>
- Google Antigravity documentation: <https://antigravity.google/docs/home>

Final Advice

Remember: The best beginner strategy is: validate the idea first, collect requirements properly, build one small MVP workflow, test it with real users, then improve. AI can help you move fast, but launching a product still requires clarity, testing, trust, and customer feedback.